

Reflection: My First Experience in Vibe Coding

PH Disaster Preparedness & Evacuation Guide

This project was my first structured attempt at vibe coding - building software by describing intent in natural language and iterating with an AI coding assistant rather than writing every line manually. The goal was practical: a life-safety-adjacent web app that helps Filipinos get plain-language disaster guidance, find evacuation centers, and access emergency hotlines by barangay.

Vibe coding felt fastest on well-scoped prompts. When I asked for a global footer, expanded PSGC location search, or mock evacuation centers marked clearly for later replacement, the assistant could implement end-to-end changes across UI, API routes, and tests. The experience was less like following a tutorial and more like pairing with a developer who already knew the repository layout and safety rules in `.cursorrules`.

The hardest lessons came from edge cases the first prompts did not surface. Bad coordinates in open PSGC data sent the evacuation map to the wrong region; fixing that required debugging with runtime logs, not guesswork. Importing server-only modules into client components broke the build until shared constants were split out. These moments reminded me that vibe coding still demands verification - I had to run the app, reproduce bugs, and read diffs, not only accept generated code.

I also learned to think in layers of "live" versus "mock." The app can use Supabase and Claude when keys are present, but most barangays still rely on placeholder evacuation data at the nearest mapped public school. That hybrid is honest for a prototype: the UX stays useful while real LGU data is backfilled later. Deploying to Vercel and pushing to GitHub closed the loop from idea to production URL.

Overall, vibe coding lowered the barrier to shipping a credible MVP, but it did not remove responsibility for product judgment. The assistant accelerated implementation; I still owned scope, safety defaults (UNKNOWN evac status, no fabricated hazards), and acceptance testing. I would use this workflow again - especially for exploration - while keeping typecheck, tests, and manual walkthroughs as non-negotiable guardrails.

Project: disaster-prep | Live app: disaster-prep-ruddy.vercel.app | Repository: github.com/allenarendon/disaster-prep